EnvGuard - A Tool to Protect a System from Environmental Deviations

"System in Isolation Dies"



Mid Presentation

SE - 801 - Software Project Lab III

Supervised by:
Dr. Kazi Muheymin-Us-Sakib
Professor
Institute of Information Technology
University of Dhaka

Submitted by:
Abhijit Paul (1201)
Institute of Information Technology
University of Dhaka

Submitted to:
SPL 3 program committee

Institute of Information Technology
University of Dhaka

Letter of Transmittal

12 September, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **EnvGuard**: A tool to protect a system from environmental deviations

Sir,
I am submitting the technical report for our Software Project Lab 3, "**EnvGuard**: A tool to protect a system from environmental deviations".
This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Abhijit Paul
BSSE-1201
Institute of Information Technology
University of Dhaka

01.12.2024
—----------
Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project "EnvGuard: A tool to protect a system from environmental deviations". I would like to convey my tremendous gratitude to my supervisor, Dr. Kazi Muheymin-Us-Sakib, Professor, Institute of Information Technology, University of Dhaka, for providing me with guidance and support throughout the project. His expertise and insights were instrumental in shaping the direction of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

Environment changes constantly. When designing a system, designers make certain assumptions about the system. However, systems may deviate from expected scenarios. Additionally, considering more models of environment when designing a system can make it more robust. However, designing systems for many different environmental conditions and exploring design alternatives is a time-consuming and costly task.

EnvGuard addresses this challenge by automatically generating designs based on environmental conditions. The designs are based on mathematical foundations. While existing works on formal specifications focus on mathematical specifications, Itake in one step further and incorporate UML inputs. Additionally, EnvGuard performs ranking on the generated designs to present them from simple to complex order to user, thus making exploring design alternatives 39% faster based on our study.

One of the key features of EnvGuard is that it will seamlessly integrate with agile workflow due the simplicity of using the tool and its supporting UML design. It also ensures that the generated redesigns satisfy safety property under deviated environmental conditions. EnvGuard performance is however constrained by the literature. This tool can help designers make more robust designs, thus reducing the cost of re-engineering and maintenance in the long run due to changes in environmental assumptions and requirements.

List of Tables
1. Introduction 1
1.1 Motivation 1
1.2 Scope 2
1.3 Project Timeline 2
1.4 Purpose of Document 2
2. Project Description 3
2.1. Requirements Gathering 3
2.2. Quality Function Deployment 4
Normal Requirements 4
Expected Requirements 4
Exciting Requirements 4
2.3 Usage Scenario 5
Project Creation and Management 5
Generation of Robust Designs 5
Detailed Reports and Insights 5
Customization and Configuration 5
3. Scenario Based Modeling 6
3.1 Use Case Diagram 6
Level 0 of Use Case Diagram 6
Level 1 of Use Case Diagram 7
4. Data Based Modeling 8
5. Class-Based Modeling 10
5.1 Analysis Classes 10
5.2 Class cards 10
6. Engineering - Design Ranking 16
7. Architectural Design 18
7.1 Architectural Context Diagram 18
7.2 Top Level Components (TLC) 18
7.3 Instantiation of the Architecture 19
7.4 Mapping Requirements to Software Architecture 21
7. 5 Elaborating Deployment Diagram 22
8. Preliminary Test Plan 23
8.1 High Level Description of Testing Goals 23
8.2 Test Cases 23
Test Case 1: OOB 23
Test Case 2: Project Creation 23
Test Case 3: Project Management 23
Test Case 4: Invalid File Upload 24
Test Case 5: Robust Design Generation 24

Test Case 6: Design Ranking....24
Test Case 7: Report Generation.... 25
Test Case 8: Version Control for Reports....25
Test Case 9: LLM Insights....25
Test Case 10: JSON Configuration Customization.... 25
Test Case 11: Security Testing....26
Test Case 12: Stress Testing.... 26
Conclusion.... 26
References.... 27

List Of Figures

Figure 1. Therac-25 System Design 1
Figure-2: Project Timeline (Estimated) 2
Figure 3: Use Case Diagram L-0 6
Figure 4: Use Case Diagram L-1 7
Figure-5: ER-Diagram of EnvGuard 8
Figure-6: CRC Diagram of EngGuard 15
Figure 7. Methodology to evaluate the design metrics on human comprehension benchmark 16
Figure-8: Architectural Context Diagram 18
Figure 10. Architecture Instantiation on component Design Robustifier 19
Figure 11. Architecture Instantiation on component ProjectManager 20
Figure 12. Architecture Instantiation of EnvGuard 20
Figure 13: Instantiating Components into Artifacts and Deploying them 22

List of Tables

Table-1: Schemas of EnvGuard 9
Table-2: CRC Card (Project) 10
Table-3: CRC Card (ProjectManager) 11
Table-4: CRC Card (UML-LTLParser) 11
Table-5: CRC Card (SpecValidator) 12
Table-6: CRC Card (ReportGenerator) 12
Table-7: CRC Card (DesignRanker) 12
Table-8: CRC Card (DesignRobustifier) 13
Table-9: CRC Card (User) 13
Table-10: CRC Card (LLMInterface) 13
Table 11 : Statistical Correlation between Design Metrics and Human Comprehension Rankings 17
Table-13: Mapping of Components with Requirement 21

1. Introduction

Software is everywhere. In cyber-physical systems, critical infrastructures and high-stake systems (e.g. spacecrafts, blockchain), bugs can result in significant money loss to accidents. So it is crucial to ensure that systems satisfy critical requirements under environmental deviations. Often, bugs caused by environmental deviations occur from design level. When designing, I consider a simple model of the environment as it's not possible to consider all possible factors. Under deviations, critical requirements often are not satisfied.

A system that is robust would ensure that critical requirements are satisfied even under possible deviations in the environment. And EnvGuard is a tool to help system engineers create robust systems by considering system design*, environmental factors, and human factors. It generates multiple robust design alternatives, ranked in ascending order based on their design comprehension score. System engineers can review these redesigned options to gain valuable insights to make their system robust.

1.1 Motivation

In practice, the environment rarely remains fixed, and often deviates from its expected behavior over time [1]. Many unexpected situations occur in deviated environments. Therac-25 is a famous example of it (Figure 1). 3 people died due to radiation [1] because the beam fired before switching when the operator switched mode and pressed “Enter” briefly (deviated user action). So it is necessary to consider the environment when designing a system from its specification. Additionally, safety-critical systems are not uncommon. Most cyber-physical system e.g. IoT), XRay user interface, security mechanisms, e-voting systems are critical and often built in the traditional software industry. These projects can greatly benefit from consideration of the environment during the design phase. Ignoring a bug at design phase will cost a lot later on.

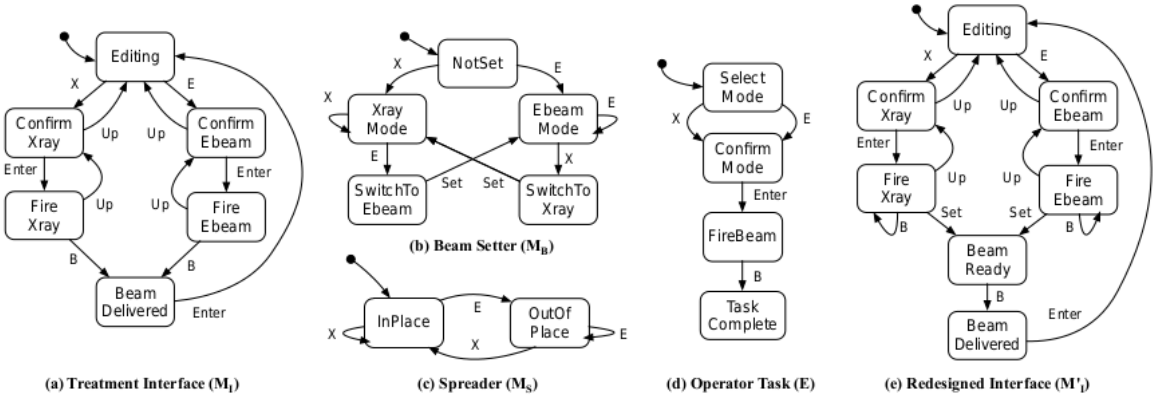


Figure 1. Therac-25 System Design

With increasing reliance on softwares and cyber physical systems working under real world conditions, it is absolutely essential to build robust softwares. Because if the software controls a real world physical device, its bug can cause accidents (e.g. autonomous vehicles).

1.2 Scope

Stakeholders preferred a system that is easily adaptable in Agile. So the following specifications are within the scope of this project.

1. Take both UML diagrams and LTL specifications as input.
2. Convert UML to LTL using the approach proposed in [8].
3. Take cost, preferred behaviors, controllable and observable events in JSON format, as used in [1].
4. Generate redesigns using Supervisory Controller Synthesis proposed in [1].
5. Convert the redesigns back to UML using [8].
6. As an exciting task, ranking designs based on calculated design comprehension score.
7. Train a model to predict the design comprehension score of behavioral models based on environmental factors.

There are however certain **limitations** in our project. I consider them out-of-scope for now.

1. The paper [1] used 3 Safety-Critical systems to evaluate their approach. I can extend this number by at most 2. Because defining complex real world systems formally is beyond the scope of our work due to inherent complexity and time constraints.
2. Not all systems can be expressed formally using UML humanely.
3. I could find no previous work on predicting design comprehension of behavioral models based on human factors. So I leverage similar works in literature to do it [9]. The performance is constrained by literature.

1.3 Project Timeline

The estimated timeline of EnvGuard is given below:

Task	Phase 1				Phase 2				Phase 3			
	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4
Prerequisite Knowledge	-											
Requirements Specification												
Project Design												
Project Development												
Project Testing												
Final Report Writing												

Figure-2: Project Timeline (Estimated)

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

The following depicts the overview of the quality function deployment of EnvGuard. The journey towards defining EnvGuard's Quality Function Deployment (QFD) began with an in-depth review of relevant research literature, notably the papers authored by Eunsuk Kang's research group [1], providing valuable insights into the domain. This was complemented by proactive stakeholder engagement, involving key players in the software development and design sectors, to thoroughly understand their perspectives and concerns. System engineers and senior software developers from a wide range of software farms were stakeholders of the project.

2.1. Requirements Gathering

Our primary stakeholders are System Engineers and Software Engineers. Our secondary stakeholders are software development farms. I first talked with Samsung R&D practitioners and observed a lack of formalization in developing safety-critical softwares. I asked why that is the case. The answer was that - formalization takes too much time. The available tools also require extra training. They are not suitable for agile.

I then did literature review to engineer a solution from a technical point of view. I found lightweight formal verification along with UML support to work for agile. I then talked with 3 software engineers from Brain Station and 1 software engineer from CELISE. They recommended to focus on design level formalization instead of code level formalization due to industry needs. With that in mind, I did another round of literature review and finally engineered a solution based on [1], [2]. I showed our engineered solution to the stakeholders and collected the following requirements to build a system that formally verifies a design and makes it robust.

Brain Station Software Engineer:

- The tool should provide designs that satisfy the requirements under environmental deviations
- Automated redesigns or codes are not often usable. So the tool should provide multiple redesigns so that we can go through them and decide on how to update our design, ourselves.
- The redesigns should be ranked according to their complexity for ease of navigation.
- We should be able to easily handle customizable options, using simple json.
- The tool should generate a report so that we can disseminate our findings among our team members easily.

Cefalo Software Engineer:

- I want to separate my projects for easier navigation. So each project should have separate specification and reporting features.
- There should be basic forms of version controlling available.
- The tool should be easy to use.

We had no conflicting requirements between stakeholders. So I skip the "working towards collaboration" section.

2.2. Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified-

Normal Requirements

RQ-01. Users can create new projects, view a list of existing projects, and manage project details.

RQ-02. Users can upload safety properties, system and environment specifications in UML and LTL formats for each project.

RQ-03. Users can rerun previously saved configurations for a given project.

RQ-04. Users can specify custom parameters using a simple JSON configuration file.

RQ-05. The system generates robust designs that take into account the uploaded environment specifications and safety properties.

RQ-06. Users can view and download detailed reports on the generated redesigns.

Expected Requirements

RQ-07. Users can manage project details, such as names, descriptions, and related specifications, in an organized dashboard.

RQ-08. The system validates the input files (UML and LTL formats) for compatibility before generating robust designs.

RQ-09. The system automatically ranks generated designs based on a human comprehension score, allowing users to easily identify the most intuitive redesigns.

RQ-10. The interface for uploading, managing, and editing system, environment, and safety properties is straightforward and user-friendly.

RQ-11. The system supports versioning via report documents, allowing users to keep track of changes to their projects and configurations over time.

Exciting Requirements

RQ-12. An LLM evaluates the generated redesigns to provide insights into their readability and complexity, making it easier for users to choose the most suitable options.

2.3 Usage Scenario

“EnvGuard” is a web-based tool designed for system and software engineers to create, analyze, and optimize robust designs for safety-critical systems, such as cyber-physical systems and critical infrastructures. It helps ensure that key requirements remain satisfied, even under deviations in environmental conditions.

Project Creation and Management
Users begin their experience with EnvGuard by creating new projects, where they enter essential information, including project names, descriptions, logo and relevant specifications through a streamlined dashboard. This intuitive interface facilitates an efficient setup process, allowing engineers to concentrate on design elements rather than administrative tasks. As engineers progress, they have the capability to upload system, environment, and safety properties in UML and LTL formats. EnvGuard ensures that these specifications undergo rigorous validation for compatibility, thus safeguarding the integrity of the project. The user can edit project details anytime. Note that, users need to register their account and login first.

Generation of Robust Designs
Once the specifications are uploaded, EnvGuard analyzes the data, parses UML, if any, to LTL format and generates multiple robust design alternatives. These alternatives are meticulously crafted to accommodate potential deviations in environmental conditions, thereby ensuring that critical requirements are satisfied even under unpredictable scenarios.

Detailed Reports and Insights
To further support engineers in their design endeavors, EnvGuard provides comprehensive reports that detail the robustness of the generated designs. Each design is ranked based on a design comprehension score, which enables engineers to quickly identify the most intuitive and effective options. This ranking feature not only streamlines the decision-making process but also enhances the overall user experience. Users can easily download these reports for further analysis, ensuring that they have all necessary information at their disposal for informed decision-making. Users can also see LLM generated insights with each redesign.

Customization and Configuration
Customization is a fundamental aspect of EnvGuard's functionality. Engineers can specify various behaviors of environment variables and safety requirements using straightforward JSON configuration files, tailoring the analysis to their specific project needs. This level of adaptability extends to the design iteration process, where engineers can tweak parameters for each version, ensuring that every redesign is finely tuned for optimal robustness.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

Level 0 of Use Case Diagram

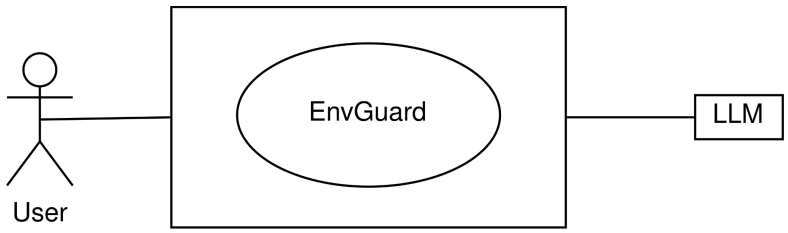


Figure 3: Use Case Diagram L-0

Name: EnvGuard
Primary actors: User
Secondary actors: LLM
Goal in Context: The diagram shown above represents the whole tool for protecting a system from environmental deviations - EnvGuard. Note that LLM is the interface to the external OpenAI API.

Level 1 of Use Case Diagram

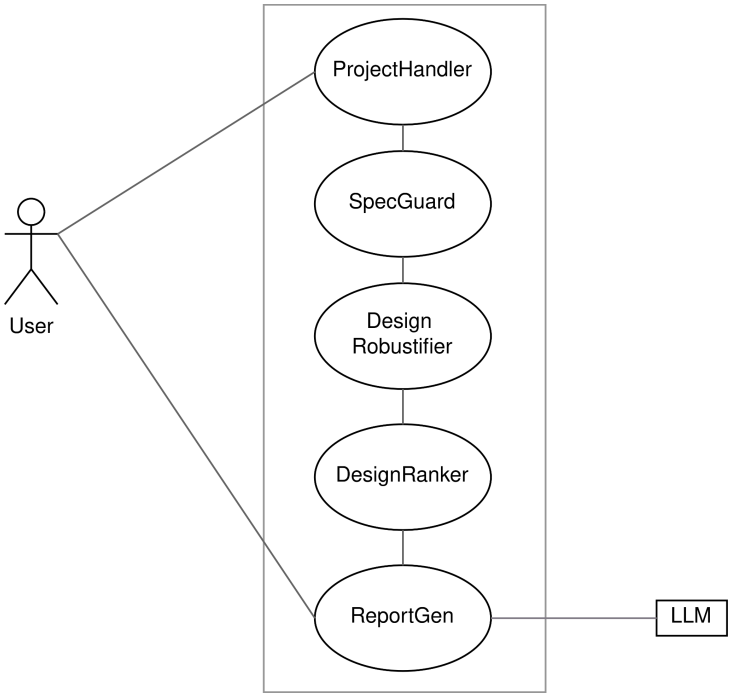


Figure 4: Use Case Diagram L-1

Name: EnvGuard
Primary actors: User
Secondary actors: LLM
Goal in Context: The diagram shows the modules of EnvGuard. EnvGuard consists of 5 modules. The modules can be elaborated as follows:

1. **ProjectHandler**: This module serves as the entry point for EnvGuard's functionality. User creates a project using the project title, description and logo. Then they upload system specification, environment specification and safety property. Additionally, they define custom parameters using a json format. At home, they can view all the available projects.
2. **SpecGuard**: This module handles UML to LTL conversions and validation. It first validates the uploaded specs. It then converts UML diagrams (if any) to LTL format.
3. **Design Robustifier**: This module focuses on the algorithmic part to generate robust designs based on given requirements for the user.
4. **DesignRanker**: The module ranks the designs based on the user's ease of comprehension.
5. **ReportGen**: This module generates the report on the generated redesigns against the configurations. It also keeps track of the different versions of the report, to enable reproducibility. It also uses LLM to generate insights on designs.

4. Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

I first identify nouns from usage scenarios. I filter nouns that are in problem space and solution space. I then determine whether each noun in solution space is a data object. attributes. A data object can be an external entity, a thing, an occurrence, a role, an organizational unit, a place or a structure. I assign these attributes to each noun. I finally decide nouns that have more than 3 attributes as a data object/entity. I use experience to refine it further. The entities of EnvGuard after analyzing the usage story are:

- Project
- ReportVersion
- User

The ER-Diagram [10] of EnvGuard can be shown as below:

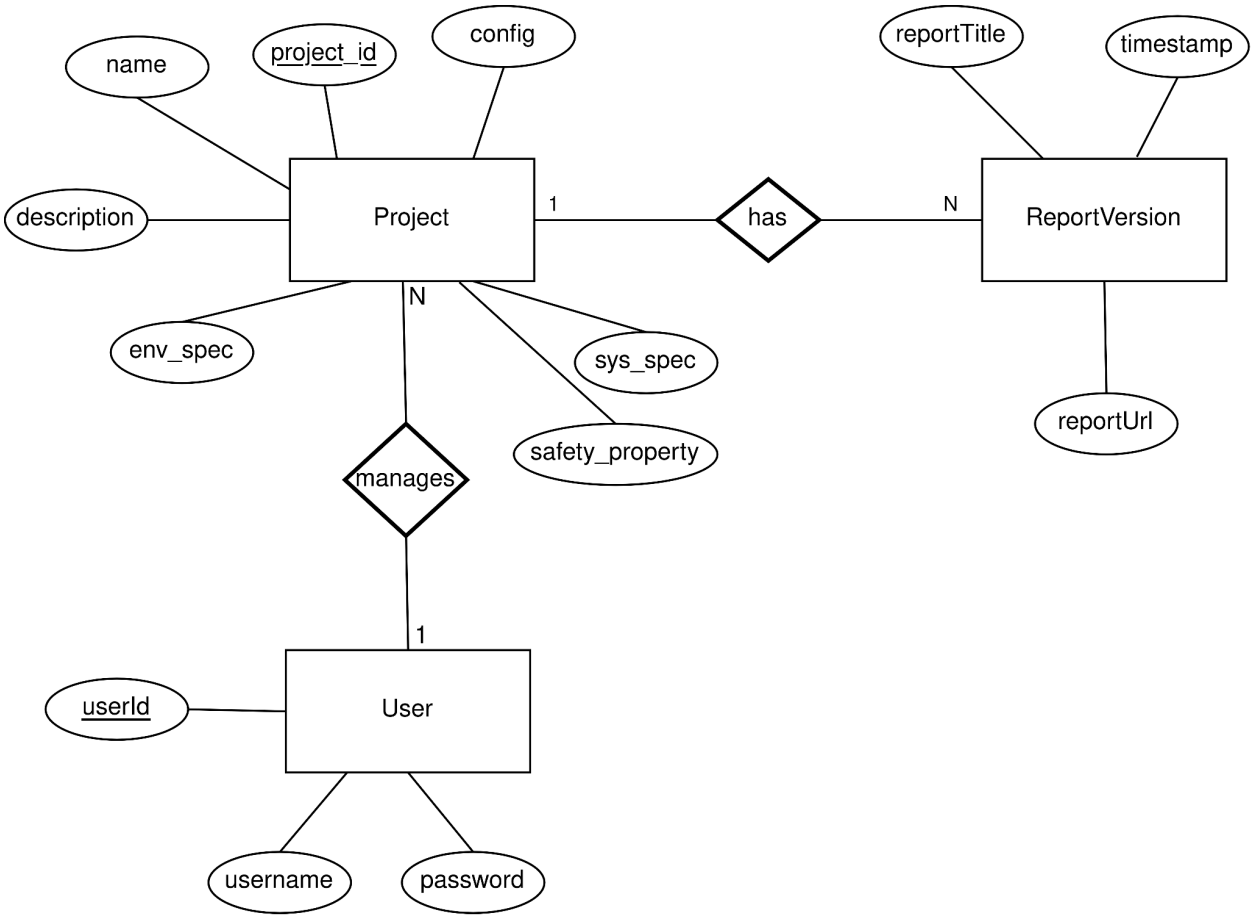


Figure-5: ER-Diagram of EnvGuard

The schemas derived from the ER-diagram is as follows:

Table-1: Schemas of EnvGuard

Schema	Attributes	Type
Project	project_id (PK)	INT
	User_id (FK)	INT
	name	varchar(30)
	description	varchar(100)
	sys_spec	Date
	env_spec	varchar(50)
	config	varchar(50)
	safety_property	varchar(50)
User	user_id (PK)	INT
	username	varchar(30)
	password	varchar(50)
Report	timestamp (PK)	Date
	project_id (FK)	INT
	report_title	varchar(50)
	report_url	varchar(50)

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

I first identified 66 nouns from the scenario. I then inspected the nouns manually, discarding ones with similar meanings e.g. POS variations. I finally had 23 distinct nouns in solution space. I then used General Characteristics (External entities, Things, Events, Roles, Organizational units, Places, and Structures). I ended up with 15 potential classes. I then used Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements) to select 9 analysis classes. After that, I rename the classes appropriately for better understanding. Each step is included in the supplementary document. The selected 9 classes are:

1. ProjectManager
2. Project
3. UML-LTLParser
4. SpecValidator
5. ReportGenerator
6. DesignRanker
7. DesignRobustifier
8. User
9. LLMInterface

5.2 Class cards

The class cards of EnvGuard is given below:

Table-2: CRC Card (Project)

Project	
Attributes	**Methods**
-project_name -description -logo -system_spec -environment_spec - safety_spec - json_config - version	+getter() +setter() -validate_input()
Responsibilities	**Collaborators**

• Users can create new projects, view a list of existing projects, and manage project details. • Users can specify custom parameters using a simple JSON configuration file.	ProjectManager, SpecValidator

Table-3: CRC Card (ProjectManager)

ProjectManager	
Attributes	**Methods**
- projects - reports - user_dashboard	+create_project() +edit_project() +delete_project() +get_project_list() +save_configuration()
Responsibilities	**Collaborators**
• Manages project creation and overall project organization. • Handles actions related to adding, deleting, and managing projects. • Aggregates and manages project details and report versions.	Project, User, DesignRobustifer

Table-4: CRC Card (UML-LTLParser)

UML-LTLParser	
Attributes	**Methods**
- uml_data - ltl_data	+convert_uml_to_ltl() +convert_ltl_to_uml()
Responsibilities	**Collaborators**
• Converts UML data into LTL format for further analysis. • Ensures compatibility of specifications for robust design generation.	DesignRobustifier

Table-5: CRC Card (SpecValidator)

SpecValidator	
Attributes	**Methods**
- system_spec - environment_spec - safety_spec	+validate_system_spec() +validate_environment_spec() +validate_safety_spec()
Responsibilities	**Collaborators**
● Validates UML and LTL specification files for compatibility. ● Ensures integrity of input files before analysis.	Project

Table-6: CRC Card (ReportGenerator)

ReportGenerator	
Attributes	**Methods**
- report_data - design_data - report_versions	+generate_report() -compile_results() -export_report() + manage_versioning()
Responsibilities	**Collaborators**
● Creates detailed reports of generated robust designs. ● Provides insights into the robustness of each design iteration. ● Manages versions of reports, keeping track of design iterations.	DesignRobustifier, LLMInterface

Table-7: CRC Card (DesignRanker)

DesignRanker	
Attributes	**Methods**
- design_scores - ranking_criteria - redesign_data	+rank_designs() +calculate_score()
Responsibilities	**Collaborators**

● Ranks robust design alternatives based on comprehension score. ● Facilitates easy identification of the most intuitive designs.	DesignRobustifier

Table-8: CRC Card (DesignRobustifier)

DesignRobustifier	
Attributes	**Methods**
- robust_designs - input_data	+generate_alternatives() +adjust_parameters()
Responsibilities	**Collaborators**
● Produces multiple robust design alternatives. ● Accommodates deviations in environmental conditions for robust designs.	DesignRanker

Table-9: CRC Card (User)

User	
Attributes	**Methods**
- user_id - username - password - email - profile_info	+register() +login() +logout() +update_profile() +reset_password() - access_control()
Responsibilities	**Collaborators**
● Manages user accounts, including registration and authentication.	ProjectManager

Table-10: CRC Card (LLMInterface)

LLMInterface	
Attributes	**Methods**
	+getResponseFromLLM()
Responsibilities	**Collaborators**

An LLM evaluates the generated redesigns to provide insights into their readability and complexity, making it easier for users to choose the most suitable options.	ReportGenerator

The CRC diagram of the classes is given below. I follow diagram syntax according to [5], [6].

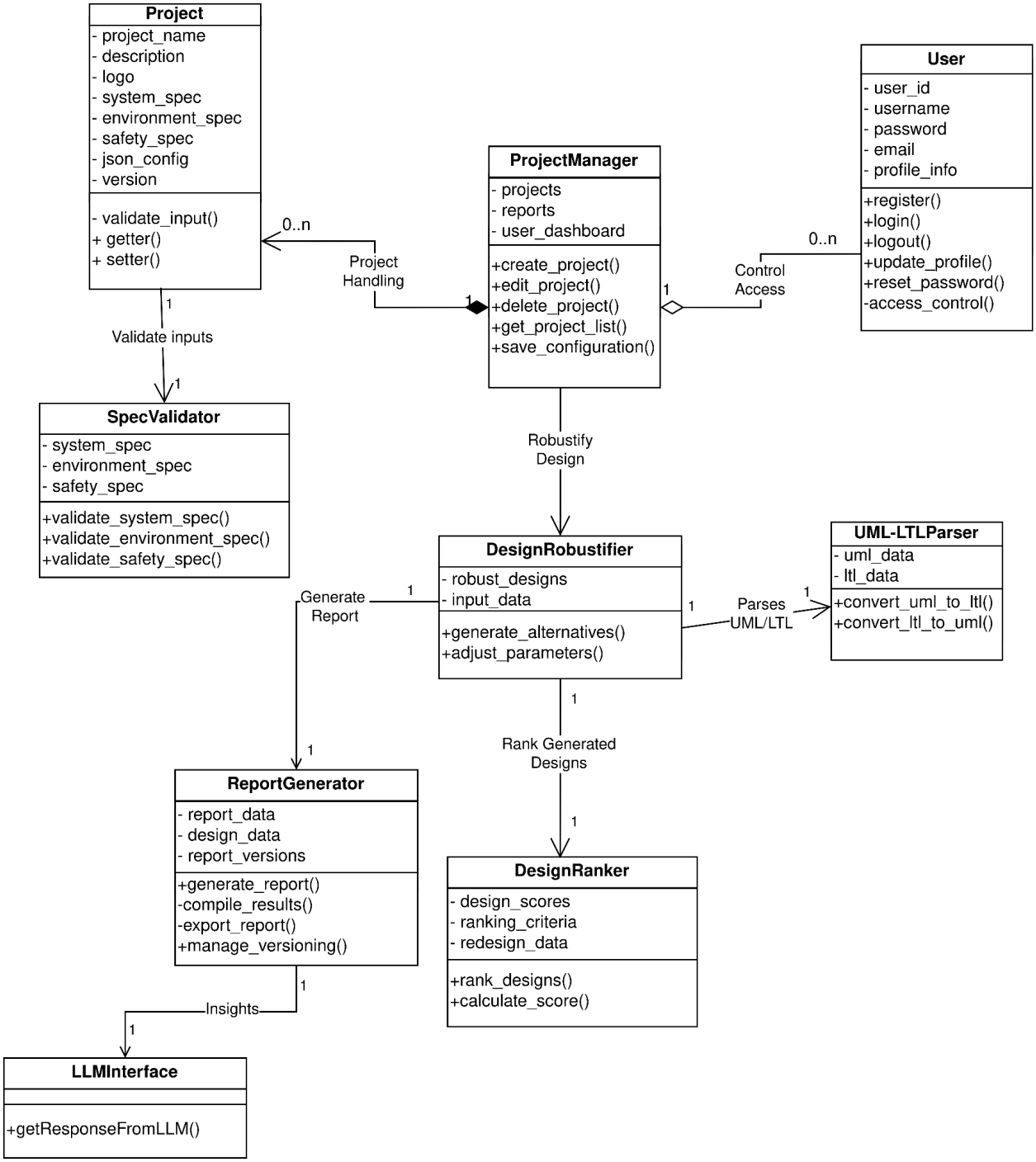


Figure-6: CRC Diagram of EngGuard

6. Engineering - Design Ranking

Design Ranking was a core requirement of stakeholders. But I could find no previous literature that ranks formally specified designs. To address this, I performed engineering on ranking formally specified designs. The details and references can be found in the original paper submitted in **ICSE Design'25** Workshop for further peer-review [7].

Our study [7] addresses these gaps by investigating design metrics that can effectively approximate human comprehension of software designs. Specifically, I focus on Labeled Transition System (LTS) designs, widely used in prominent model-checking tools like SPIN, NuSMV, LTSA, and design repair tools such as Fortis. I evaluate design complexity and structural metrics, using measures such as Albin complexity, cyclomatic complexity, and state space size for complexity, and redundancy, modularity, average branching factor, and maximum depth for structural evaluation.

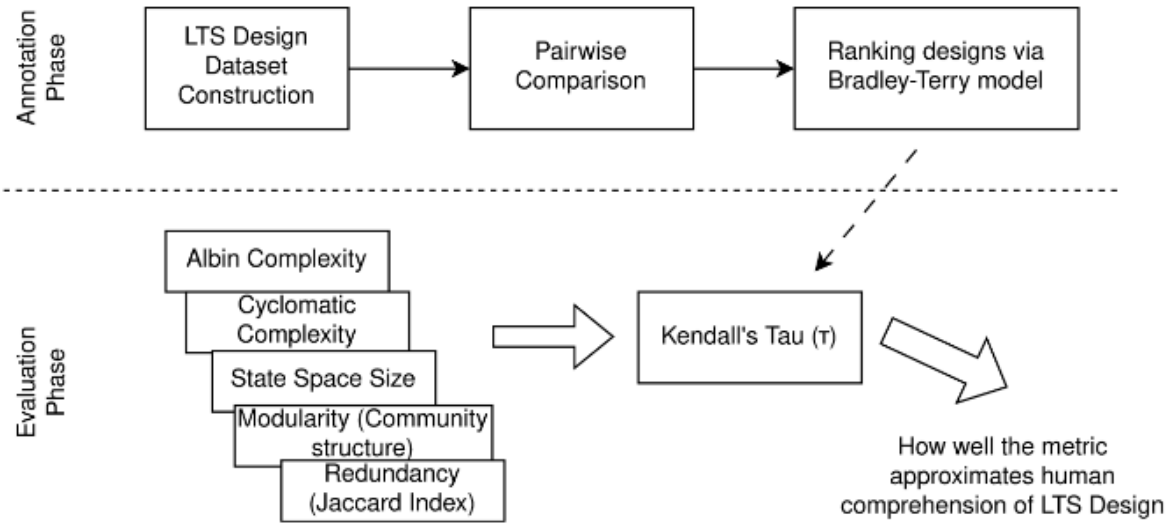


Figure 7. Methodology to evaluate the design metrics on human comprehension benchmark

To assess the effectiveness of these seven metrics in approximating human comprehension, I constructed a dataset of LTS designs derived from Kramer's book [8] and redesigns generated by Fortis. The dataset initially contained 148 LTS designs. Recognizing that ranking designs through pairwise comparisons provides improved reliability over ordinal classification [9], I performed pairwise comparisons on 48 randomly sampled designs, as comparing all (1482 pairs was infeasible. Three annotators evaluated 324 pairs of designs, sampled from 1128 pairs. The pairwise comparison agreement score between the annotators was 75.748%. The resulting comparison data was then used to rank the designs using the Bradley-Terry model. I measured the correlation between metric-based rankings and human-derived rankings using Kendall's Tau. Results indicate that Albin complexity, state space size, redundancy, and cyclomatic complexity align most effectively with human comprehension of LTS designs.

Design Metric	Kendall's Tau (τ)	P-value
Cyclomatic Complexity (V) [4]	0.3656565657	0.0003983119023
State Space Size [5]	0.4202020202	4.71E-05
Average Branching Factor	0.09898989899	0.3377263584
Max Depth	0.4202020202	4.71E-05
Albin Complexity [3]	0.4444444444	1.68E-05
Modularity (Q) [31]	-0.2363636364	0.02207570243
Redundancy (J) [6]	0.3151515152	0.00227258672

Table 11 : Statistical Correlation between Design Metrics and Human Comprehension Rankings

To validate our findings, I conducted a case study. I implemented the best-performing metric, Albin complexity, on top of Fortis and generated 16 repaired designs for a simple voting system described in [12]. To evaluate the impact, I divided the redesigns into two groups: one where the 16 designs were presented in their original order and another where the designs were ranked based on Albin complexity to align with human comprehension. Two annotators then explored these designs to understand the repairs. The annotator who explored the ranked designs, progressing from simpler to more complex designs, completed their exploration 39% faster.

The co-authors, Dr. Kazi Sakib and Proma Chowdhury, have reviewed the paper. This serves as our preliminary degree of validation. I hope for further validation of our methodology from ICSE'25 Design peer-review process. I implement the best performing 4 metrics, namely Albin complexity, Cyclomatic complexity, Newman modularity, Jaccard similarity index in the current version of this tool. The details of implementation are discussed in the upcoming section.

7. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

7.1 Architectural Context Diagram

The architectural context diagram for the tool is given below-

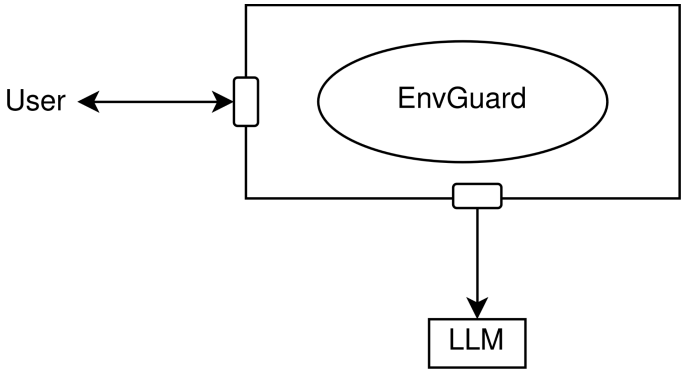


Figure-8: Architectural Context Diagram

The target for the tool have the same roles, albeit with diverse demography (Senior Software Engineer, System Engineer). Due to their similarity in using our tool, I combine them into one generic target user group - User. They are software developers and researchers. Both types of users are the producers and also consumers of the information through the tool. There is one subordinate system, LLM. It provides processing that is necessary to complete target system functionality.

7.2 Top Level Components (TLC)

Overall architectural structure with top level components is illustrated below. Note that, Top Level Components diagrams show a very high level view of our system. It is further refined into architecture in the following sections. I keep the top level diagrams to 2 independent responsibilities due to the inherent smaller size of the system and for ease of understanding the system at higher level.

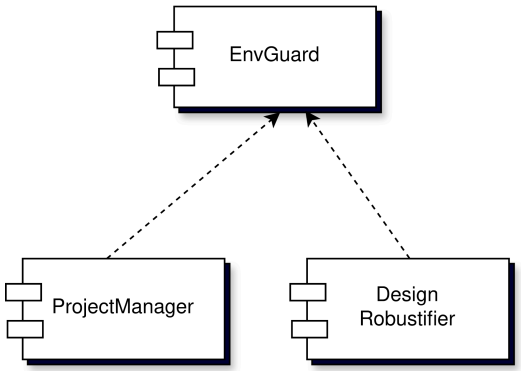


Figure-9: Top Level Components

7.3 Defining Archetypes & Instantiation of the Architecture

The components identified in Section-7.2 are broad. They need to be refined into smaller components. And just a collection of components doesn't create a meaningful design. So I apply architecture on the components to make a maintainable collection of components. The architecture is applied to a specific problem with the intent of demonstrating that the structure and components are appropriate [4].

Component: Design Robustifier

The Pipe and Filter architecture is a software design pattern that organizes a system as a series of processing elements (filters) connected by communication channels (pipes). It is particularly useful for designing applications that involve processing streams of data in a linear and sequential manner. Each filter performs a specific task on the incoming data and passes the results to the next filter through the pipes, creating a data flow pipeline.

1. **Filter:** All processes that contain a significant general algorithm that can be reused in other tasks. The filters can be re-ordered to solve different problems.
2. **Pipe:** Specific tasks that are required to communicate between the filters.

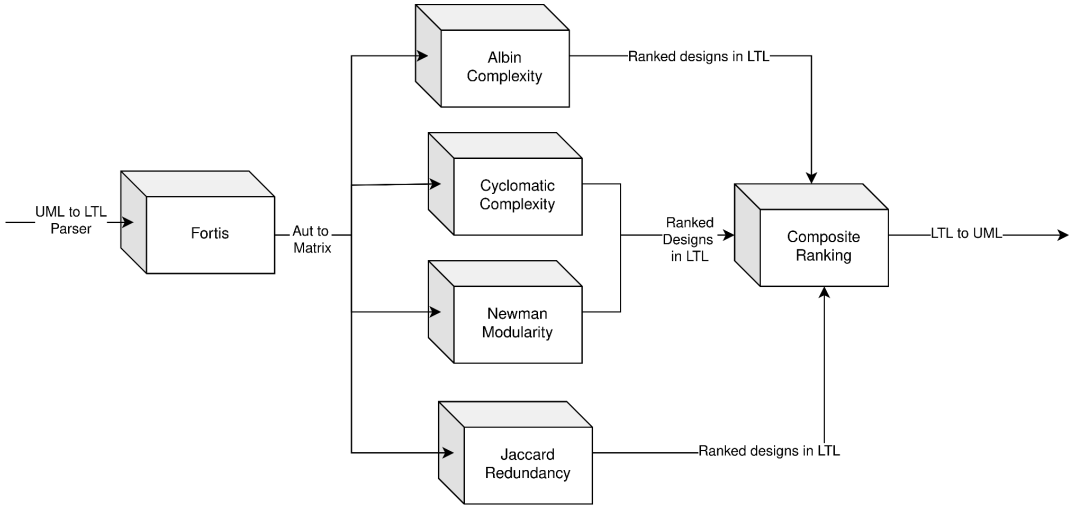


Figure 10. Architecture Instantiation on component Design Robustifier

Component: ProjectManager

Layered architecture is a software design pattern that organizes a system's components into distinct layers, with each layer serving a specific set of functionalities and having well-defined interfaces for communication. It is commonly used in various applications, from web development to enterprise software, due to its clear separation of concerns and ease of maintenance.

1. **Presentation Layer**: This layer gets and presents data from and to the system from the user.
2. **Business Logic Layer:** Contains the main business logic of our system.
3. **Utility Layer**: It contains miscellaneous functions that are used by the business logic layer. It includes library functions, frameworks and well-established algorithms.

4. **Persistence Layer:** It contains interfaces to document access objects.

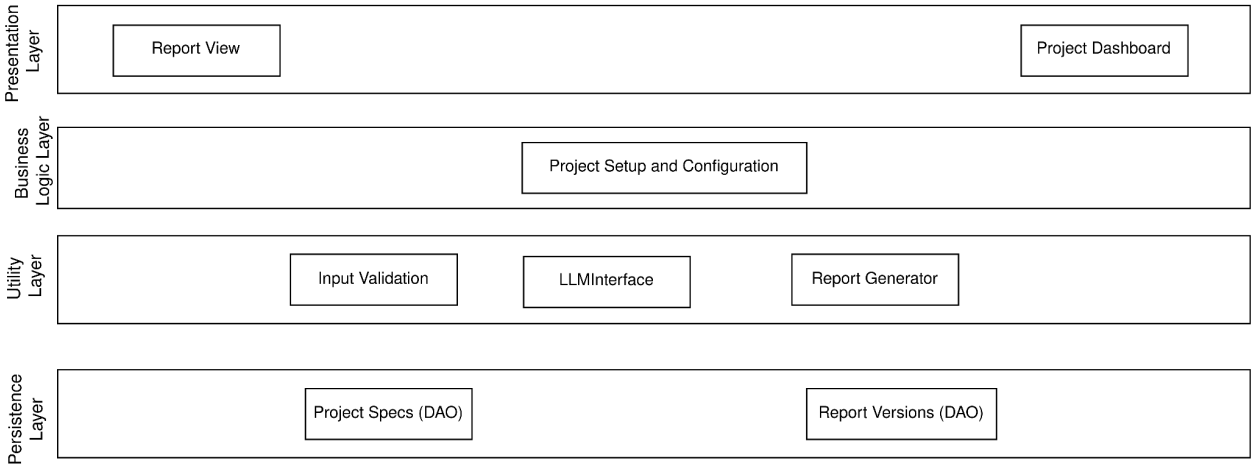


Figure 11. Architecture Instantiation on component ProjectManager

Combined Architecture of EnvGuard. I combine the two TLC into EnvGuard architecture.

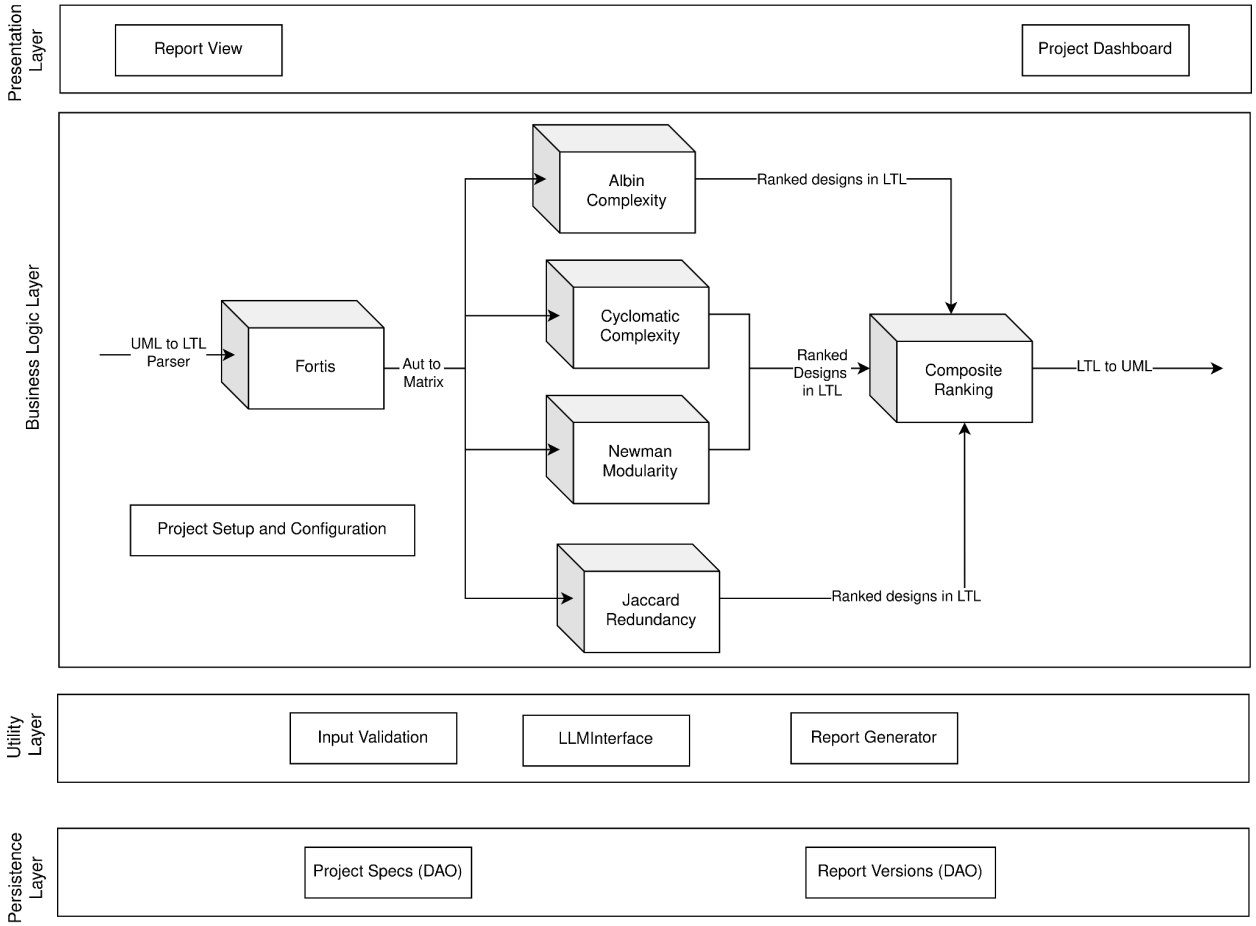


Figure 12. Architecture Instantiation of EnvGuard

7.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown as below:

Table-13: Mapping of Components with Requirement

Requirements	**Components**
-Users can create new projects, view a list of existing projects, and manage project details. -Users can manage project details, such as names, descriptions, and related specifications, in an organized dashboard.	Project Dashboard, Project Spec (DAO)
-Users can upload safety properties, system and environment specifications in UML and LTL formats for each project. -Users can rerun previously saved configurations for a given project. -Users can specify custom parameters using a simple JSON configuration file. -The interface for uploading, managing, and editing system, environment, and safety properties is straightforward and user-friendly.	Project Setup and Configuration, Project Spec (DAO)
The system validates the input files (UML and LTL formats) for compatibility before generating robust designs.	Input Validation
The system generates robust designs that take into account the uploaded environment specifications and safety properties.	Fortis
The system automatically ranks generated designs based on a human comprehension score, allowing users to easily identify the most intuitive redesigns.	Albin Complexity
	Cyclomatic Complexity
	Jaccard Redundancy
	Newman Modularity
	Composite Ranking
	Aut to Matrix
-Users can view and download detailed reports on the generated redesigns. -The system supports versioning via report documents, allowing users to keep track of changes to their projects and configurations over time.	Report Generator, Report View, Report Version DAO

-An LLM evaluates the generated redesigns to provide insights into their readability and complexity, making it easier for users to choose the most suitable options.	LLMInterface
Robustification of Design	Fortis, UML to LTL parsing, LTL to UML Parsing

7. 5 Elaborating Deployment Diagram

First, a deployment diagram is built. Then at a later phase, it is refined into instance form - that is, instantiating each deployment unit with created artifacts etc. According to UML2.0 Deployment specification and Pressman(Page 248-249). I refine the components into the following instance form for deployment.

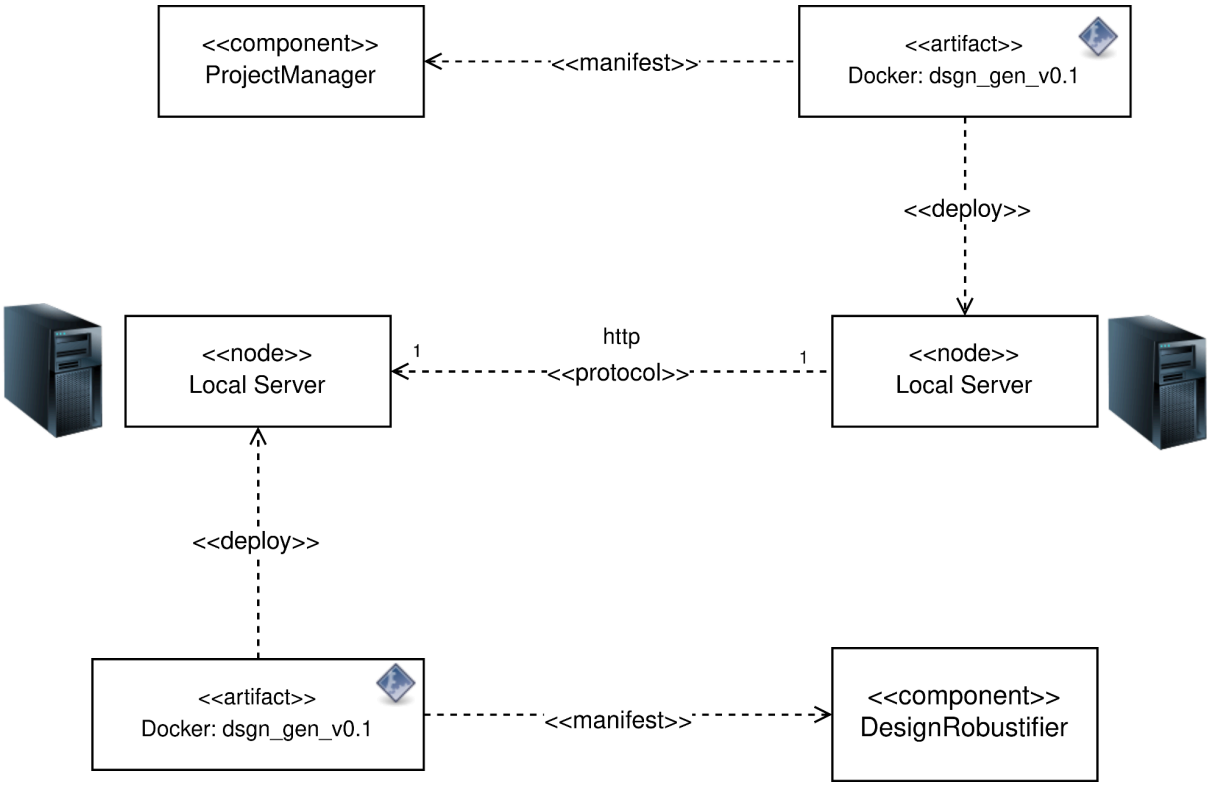


Figure 13: Instantiating Components into Artifacts and Deploying them

In UML 2.0 Deployment Diagrams, the focus is on the **physical deployment** of artifacts to nodes and the communication between nodes. So I skip the LLM requests since I will use OpenAI API for it and I won't do physical deployment for LLM.

8. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

8.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To demonstrate that the tool meets its requirements
- To ensure the features of npm packages are are extracted correctly
- To ensure that it produces correct detection most of the time
- To ensure the false positive and negative results are mitigated
- To ensure that it provides detailed analysis report
- To ensure that the tool receives feedback from the users properly
- To ensure that the tool runs smoothly

8.2 Test Cases

Test cases for the testing of this tool are documented below-

Test Case 1: OOB

Test Scenario: OOB means out of box experience. It is extremely necessary for the user to use a product.
Precondition: The user does not have any account yet.
Test Steps:

1. Click “Register” and enter necessary information
2. Enter Dashboard

Expected Result: The dashboard should lead to a documentation/demo on how to use the tool.
Module: Project Dashboard
Requirement ID: RQ-10

Test Case 2: Project Creation

Test Scenario: Create a new project by entering all required fields (name, description) and uploading system/environment specifications in UML and LTL formats.
Precondition: The user is registered, logged in, and on the project dashboard.
Test Steps:

3. Click "Create New Project" and fill in required fields.
4. Upload UML and LTL files and submit.

Expected Result: The project is successfully created, and its details are visible in the project dashboard.
Module: Project Dashboard, Project Spec (DAO)
Requirement ID: RQ-01

Test Case 3: Project Management

Test Scenario: Edit the project details, such as name, description, and related specifications.

Precondition: A project has been created and is listed on the dashboard.
Test Steps:

1. Select a project
2. Edit details
3. Click Save.

Expected Result: The updates are successfully saved, and the edited details are displayed on the project dashboard.
Module: Project Dashboard, Project Spec (DAO)
Requirement ID: RQ-02

Test Case 4: Invalid File Upload

Precondition: A project is created and selected for editing.
Test Scenario: Upload invalid or incompatible UML and LTL files.
Test Steps:

1. Open the project and navigate to the "Upload Properties" section.
2. Upload an invalid UML or LTL file and confirm

Expected Result: The system displays an error message, indicating the file incompatibility.
Module: Input Validation
Requirement ID: RQ-04

Test Case 5: Robust Design Generation

Precondition: A project has valid UML and LTL specifications uploaded.
Test Scenario: Generate robust designs based on the uploaded specifications.
Test Steps: Navigate to the "Generate Designs" section and initiate the process.

Expected Result: The system generates multiple design alternatives, considering deviations in environmental conditions.
Module: Fortis
Requirement ID: RQ-05

Test Case 6: Design Ranking

Precondition: Robust designs are generated for a project.
Test Scenario: View the automatically ranked designs based on human comprehension scores.
Test Steps: Navigate to the "Design Ranking" section and review the ranked designs.

Expected Result: The system ranks the designs and displays the most intuitive redesigns at the top.
Module: Albin Complexity, Cyclomatic Complexity, Jaccard Redundancy, Newman Modularity,

Composite Ranking
Requirement ID: RQ-06

Test Case 7: Report Generation

Precondition: Robust designs are generated and ranked for a project.
Test Scenario: Generate and download a detailed report for the selected design.
Test Steps:

1. Navigate to the "Generate Report" section.
2. Select a design and download the report.

Expected Result: The system generates a comprehensive report and allows the user to download it.
Module: Report Generator, Report View
Requirement ID: RQ-07

Test Case 8: Version Control for Reports

Precondition: A project has multiple reports generated over time.
Test Scenario: View the version history of reports for a project.
Test Steps:

1. Navigate to the "Report History" section.
2. Select and view previous versions.

Expected Result: The system displays a list of all report versions, allowing users to view older versions.
Module: Report Version DAO
Requirement ID: RQ-08

Test Case 9: LLM Insights

Precondition: Robust designs are generated for a project.
Test Scenario: View LLM-generated insights for a specific redesign.
Test Steps: Select a redesign and open the "Insights" tab

Expected Result: The system provides readability and complexity analysis for the selected redesign.
Module: LLMInterface
Requirement ID: RQ-09

Test Case 10: JSON Configuration Customization

Precondition: A project is created, and robust designs have not been generated.
Test Scenario: Provide custom parameters in a JSON configuration file for a redesign.
Test Steps: Upload a JSON file with custom parameters and confirm.
Expected Result: The system processes the JSON file and updates the design parameters for analysis.
Module: Project Setup and Configuration
Requirement ID: RQ-10

Test Case 11: Security Testing

Precondition: User has been registered
Test Scenario: SQL Injection attack
Test Steps: Upload specifications with SQL injection attack
Expected Result: The system should return output similar to “Wrong specification format”.
Module: Project Setup and Configuration
Requirement ID: RQ-10

Test Case 12: Stress Testing

Precondition: Backend apis can be accessed.
Test Scenario: Simulating a high volume of requests to evaluate its performance under load.
Test Steps:

1. Start backend server with 4 worker processes
2. Incrementally increase load and check the average response time.

Expected Result: The response time should be appropriate for 3 concurrent users.
Module: All
Requirement ID: All

Conclusion

In this report, I have discussed all the necessary technical details to develop the “**EnvGuard**: A System in Isolation dies”. I have discussed the scope, assumptions, QFD and usage scenario. Scenario-based modeling, data modeling and class-based modeling have been performed. The architectural design has been discussed. I have also presented the high-level description of testing goals and summary of items and features to be tested. I have added figures and tables for better understanding of the technical details.

Though EnvGuard addresses pressing issues of securing a system in an environmental context and considers human comprehension, there is further room for improvements, such as optimizing the redesigns for interpretability. With this document, I have tried to clearly state the workflow of the total system. I hope this report can help the reader have an appropriate understanding of the technical details of this tool.

References

[1] Zhang, Changjian, et al. "Robustification of behavioral designs against environmental deviations." 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023.

[2] Cartaxo, Emanuela G., Francisco GO Neto, and Patricia DL Machado. "Test case generation by means of UML sequence diagrams and labeled transition systems." 2007 IEEE International Conference on Systems, Man and Cybernetics. IEEE, 2007.

[3] Yusuf, Shehnaaz, Huzefa Kagdi, and Jonathan I. Maletic. "Assessing the comprehension of UML class diagrams via eye tracking." 15th IEEE International Conference on Program Comprehension (ICPC'07). IEEE, 2007.

[4] Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

[5] Javatpoint. UML Association vs Aggregation vs Composition. Available at: https://www.javatpoint.com/uml-association-vs-aggregation-vs-composition [Accessed Nov. 29, 2024].

[6] Bindu BC. 2018. Association, Aggregation, and Composition in OOPS. Medium. Available at: https://medium.com/@bindubc/association-aggregation-and-composition-in-oops-8d260854a446 [Accessed Nov. 29, 2024].

[7] Abhijit Paul, Proma Chowdhury, and Kazi Sakib. 2025. Towards Bridging Formal Methods and Human Interpretability: Evaluating Human Comprehension of Labeled Transition System Designs. In-review at 2nd International Workshop on Designing Software. Available at: https://designing2025.hotcrp.com/paper/5?cap=hcav5zccLkoyqGXWnQUPGwayTVeSA [Accessed Nov. 29, 2024].

[8] Adriana Sejfia, Max Schäfer. 2022. Practical Automated Detection of Malicious npm Packages. In 2022 International Conference in Software Engineering (ICSE). 1681–1692. https://arxiv.org/pdf/2202.13953.pdf

[9] Pronnoy Goswami, Saksham Gupta, Zhiyuan Li, Na Meng, and Daphne Yao. 2020. Investigating The Reproducibility of NPM Packages. In 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME). 677–681. https://doi.org/10.1109/ICSME46990.2020.00071

[10] Wikipedia. Entity–relationship model. Available at: https://en.wikipedia.org/wiki/Entity%E2%80%93relationship_model [Accessed Nov. 29, 2024].